

Equalização de Histograma e Processamento Paralelo em Java

Sistemas Multinúcleo e Distribuídos

Mestrado em Engenharia Informática – 2025/2026

Porto, maio de 2026

Luis Manuel Nazário Mendes Santos

Versão 3, 2026-05-11

Tabela de Revisões

Revisão	Data	Autor(es)	Descrição
1	2026-05-11	Luis Manuel Nazário Mendes Santos	Versão inicial do relatório
2	2026-05-11	Luis Manuel Nazário Mendes Santos	Implementações paralelas e benchmark
3	2026-05-11	Luis Manuel Nazário Mendes Santos	Análise de GC e conclusões

Conteúdo

Lista de Figuras	vi
Lista de Tabelas	vii
Declaração de Integridade	xi
1 Introdução	1
2 Objetivos	3
3 Implementações	5
3.1 Solução Sequencial	5
3.2 Multithreaded sem Thread Pool	6
3.3 Thread Pool com ExecutorService	7
3.4 Fork/Join Framework	7
3.5 CompletableFuture	8
3.6 Tuning do Garbage Collector	9
3.6.1 Garbage Collectors Avaliados	9
3.6.2 Resultados e Escolha	10
4 Concorrência e Sincronização	11
4.1 Estrutura das Fases	11
4.2 Thread Safety	11
4.3 Estruturas de Dados e Mecanismos	12
5 Análise de Performance	13
5.1 Configuração do Ambiente	13
5.2 Resultados	14
5.3 Eficiência e Ganhos	14
5.4 Escalabilidade	15
5.5 Overhead de Coordenação	15
5.6 Análise de Memória	16

Conteúdo

5.7	Bottlenecks Identificados	16
6	Conclusões	17
6.1	Síntese dos Resultados	17
6.2	Garbage Collector	17
6.3	Reflexão	17
	Referências	19

Lista de Figuras

1.1 Efeito visual da equalização de histograma 1

Lista de Tabelas

3.1	Comparação de Garbage Collectors (sequencial / melhor paralelo)	10
4.1	Mecanismos de sincronização por implementação	12
5.1	Resultados completos do benchmark (G1GC, imagem 860×1276)	14
5.2	Escalabilidade com número de threads (Thread Pool e CompletableFuture)	15

Declaração de Integridade

Declaro, por este meio, que o presente trabalho académico foi realizado com integridade.

Afirmo não ter recorrido a plágio, falsificação de resultados ou qualquer outra forma de utilização indevida de informação no decurso da sua elaboração.

Assim, o trabalho aqui apresentado é original e da minha autoria, não tendo sido previamente utilizado para outros fins que não os explicitamente reconhecidos. Este documento identifica de forma transparente o modo como foram utilizadas ferramentas de IA, bem como o propósito dessa utilização.

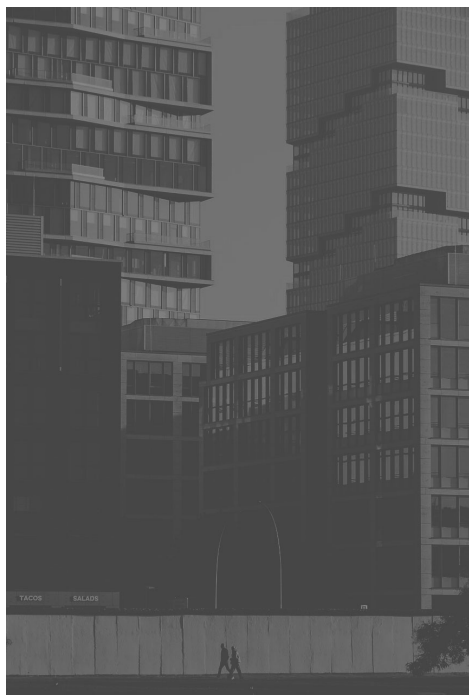
Confirmo ainda o meu conhecimento e respeito pelo Código de Conduta Ética do P.PORTO.

ISEP, Porto, 11 de maio de 2026

1 Introdução

O processamento digital de imagem é uma área com aplicações em medicina, fotografia, visão por computador e muitas outras disciplinas. Uma das operações mais fundamentais é a **equalização de histograma**, que redistribui as intensidades dos píxeis de modo a aumentar o contraste de imagens de baixo contraste.

Neste projeto, implementa-se a equalização de histograma em Java, explorando diferentes estratégias de concorrência: solução sequencial, multithreading manual, thread pools, o framework Fork/Join e o paradigma assíncrono com `CompletableFuture`. O objetivo não é apenas a correção dos resultados, mas também a análise de desempenho e escalabilidade de cada abordagem.



(a) Imagem original



(b) Imagem após equalização

Figura 1.1: Efeito visual da equalização de histograma

A imagem utilizada tem dimensões 860×1276 píxeis (1 097 360 píxeis no total), e os

1 Introdução

testes foram executados numa máquina com 22 processadores lógicos.

O algoritmo de equalização é dividido em três fases:

1. **Histograma de luminosidade:** array de 256 inteiros onde a posição i contém o número de píxeis com luminosidade i .
2. **Histograma cumulativo:** array de 256 inteiros onde a posição i contém o número de píxeis com luminosidade $\leq i$.
3. **Reescrita dos píxeis:** cada píxel é transformado segundo:

$$\text{newLuminosity} = \left\lfloor 255 \times \frac{\text{cumulativeHist}[L]}{\text{totalPixels}} \right\rfloor$$

2 Objetivos

O projeto tem como objetivos principais:

- Implementar a equalização de histograma com diferentes abordagens de concorrência em Java.
- Comparar o desempenho (tempo de execução, consumo de memória, escalabilidade) de cada implementação.
- Explorar mecanismos de sincronização e garantir *thread safety* em todas as implementações paralelas.
- Configurar e analisar o impacto de diferentes *Garbage Collectors* da JVM no desempenho da aplicação.
- Documentar e justificar as escolhas de design e concorrência em cada abordagem.

A estrutura do projeto foi organizada em pacotes Java de acordo com a responsabilidade de cada componente:

- `sismd.filters` — Interface, classe base e todas as implementações do filtro.
- `sismd.benchmark` — Infraestrutura de medição de desempenho.
- `sismd.utils` — Utilitários de I/O de imagem (código fornecido pelo docente).
- `sismd.starter` — Código de demonstração original fornecido pelo docente.
- `sismd.Main` — Ponto de entrada com menu interativo.

3 Implementações

Todas as implementações partilham uma interface comum `HistogramEqualizer` e estendem a classe abstrata `AbstractFilter`, que disponibiliza os métodos utilitários `computeLuminosity`, `buildCumulativeHistogram` e `equalizedLuminosity`. Os métodos `computeHistogram` e `applyEqualization` da `SequentialFilter` têm visibilidade de pacote (*package-private*) e são reutilizados pelas implementações paralelas como operações de folha.

3.1 Solução Sequencial

A solução sequencial serve como linha de base (*baseline*) para comparação de desempenho. Processa o array de píxeis em três passagens sequenciais correspondentes às três fases do algoritmo.

Justificação: A implementação sequencial não tem overhead de criação de threads nem de sincronização, pelo que representa o custo mínimo do algoritmo. É indispensável para calcular o *speedup* das implementações paralelas e para verificar a correção dos resultados.

Listing 3.1: Núcleo da implementação sequencial

```
public Color[][] apply(Color[][] image) {
    Color[][] result = Utils.copyImage(image);
    int totalPixels = result.length * result[0].length;

    int[] hist      = computeHistogram(result, 0, result.length);
    int[] cumulative = buildCumulativeHistogram(hist);
    applyEqualization(result, 0, result.length, cumulative, totalPixels);

    return result;
}
```

3.2 Multithreaded sem Thread Pool

Esta implementação cria e gere threads manualmente, sem reutilização. A estratégia divide as linhas da imagem igualmente entre as threads disponíveis.

Estratégia de concorrência:

1. **Fase 1 (paralela):** Cada thread calcula um histograma parcial para as suas linhas, sem qualquer sincronização — não há estado partilhado durante esta fase.
2. **Barreira:** A thread principal aguarda a conclusão de todas as threads com `join()` e faz o merge dos histogramas parciais (256 iterações, custo desprezável).
3. **Fase 2 (sequencial):** O histograma cumulativo é calculado sequencialmente ($O(256)$).
4. **Fase 3 (paralela):** Cada thread reescreve os seus píxeis — as threads acedem a linhas disjuntas, pelo que não há necessidade de sincronização.

Justificação: A ausência de *locks* ou variáveis atómicas durante as fases paralelas elimina o overhead de sincronização. A opção por histogramas parciais por thread (em vez de um array partilhado com `AtomicIntegerArray`) evita falsa partilha de cache e contention.

Listing 3.2: Fase paralela de histograma com threads manuais

```
for (int t = 0; t < actualThreads; t++) {
    final int id = t;
    final int start = id * rows / actualThreads;
    final int end = (id + 1) * rows / actualThreads;
    threads[t] = new Thread(() ->
        partialHists[id] = SequentialFilter.computeHistogram(result,
            start, end)
    );
    threads[t].start();
}
joinAll(threads); // barreira: aguarda todas as threads

int[] hist = new int[256];
for (int[] partial : partialHists)
    for (int i = 0; i < 256; i++) hist[i] += partial[i];
```

3.3 Thread Pool com ExecutorService

Esta implementação substitui a criação manual de threads por um pool fixo gerido pelo `ExecutorService`. As threads são reutilizadas entre as fases 1 e 3, eliminando o custo de criação e destruição de threads.

Justificação: Em cenários com muitas execuções do filtro (como o benchmark), o overhead de criar e destruir threads a cada execução pode ser significativo. O thread pool amortiza esse custo. As tarefas de histograma retornam `Future<int[]>`, permitindo recolher os resultados de forma limpa sem variáveis partilhadas.

Listing 3.3: Submissão de tarefas ao thread pool

```
List<Future<int[]>> histFutures = new ArrayList<>(actualThreads);
for (int t = 0; t < actualThreads; t++) {
    final int start = t * rows / actualThreads;
    final int end = (t + 1) * rows / actualThreads;
    histFutures.add(pool.submit(() ->
        SequentialFilter.computeHistogram(result, start, end)
    ));
}
int[] hist = new int[256];
for (Future<int[]> f : histFutures) {
    int[] partial = f.get();
    for (int i = 0; i < 256; i++) hist[i] += partial[i];
}
```

3.4 Fork/Join Framework

A implementação Fork/Join utiliza o paradigma de *divide-and-conquer* para dividir recursivamente o array de linhas até atingir um limiar (`THRESHOLD = 32` linhas), a partir do qual a computação é sequencial.

Estratégia:

- `HistogramTask (RecursiveTask<int[]>)`: divide o intervalo de linhas ao meio; a metade esquerda é bifurcada (`fork`), a metade direita é computada na thread atual (`compute`), e depois faz-se `join + merge`.
- `EqualizationTask (RecursiveAction)`: usa `invokeAll` para executar ambas as metades em paralelo.

Justificação: O ForkJoinPool usa *work stealing*: threads que ficam sem tarefas roubam tarefas de outras, maximizando a utilização dos processadores. O limiar de 32 linhas foi escolhido de modo a criar tarefas granulares suficientes para manter todas as threads ocupadas sem excessivo overhead de *forking*.

Listing 3.4: HistogramTask com Fork/Join

```
@Override
protected int[] compute() {
    if (end - start <= THRESHOLD)
        return SequentialFilter.computeHistogram(image, start, end);

    int mid = (start + end) >>> 1;
    HistogramTask left = new HistogramTask(image, start, mid);
    HistogramTask right = new HistogramTask(image, mid, end);

    left.fork(); // execucao assincrona
    int[] rightResult = right.compute(); // execucao sincrona
    int[] leftResult = left.join(); // aguarda left

    for (int i = 0; i < 256; i++) leftResult[i] += rightResult[i];
    return leftResult;
}
```

3.5 CompletableFuture

A implementação com CompletableFuture usa programação assíncrona declarativa para expressar a composição de tarefas sem gestão explícita de threads ou bloqueios.

Estratégia:

- `supplyAsync` lança as tarefas de histograma assincronamente num `ExecutorService` dedicado.
- `CompletableFuture.allOf(...).thenApply(...)` aguarda a conclusão de todas as tarefas e executa o merge como uma continuação não bloqueante.
- `runAsync` lança as tarefas de equalização; `allOf(...).join()` aguarda a conclusão.

Justificação: O CompletableFuture permite expressar dependências entre tarefas de forma declarativa. O uso de `thenApply` para o merge garante que este só ocorre após

todas as tarefas estarem completas, sem *polling* nem bloqueio prematuro. O `join()` sobre os futuros dentro do `thenApply` é não bloqueante porque todos já terminaram no momento em que `allOf` dispara a continuação.

Listing 3.5: Merge assíncrono com `allOf` e `thenApply`

```
int[] hist = CompletableFuture
    .allOf(histFutures.toArray(new CompletableFuture[0]))
    .thenApply(v -> {
        int[] merged = new int[256];
        for (CompletableFuture<int[]> f : histFutures) {
            int[] partial = f.join(); // nao bloqueia: todos ja concluíram
            for (int i = 0; i < 256; i++) merged[i] += partial[i];
        }
        return merged;
    })
    .join();
```

3.6 Tuning do Garbage Collector

O benchmark foi executado com quatro *Garbage Collectors* distintos da JVM para avaliar o seu impacto no desempenho da aplicação. Os logs foram gerados com a flag `-Xlog:gc:<ficheiro>.log:time,uptime,level,tags`.

3.6.1 Garbage Collectors Avaliados

Serial GC (`-XX:+UseSerialGC`): Usa uma única thread para todas as operações de GC. As pausas são stop-the-world de longa duração (até 69 ms nos logs obtidos). Não é adequado para aplicações multithreaded.

Parallel GC (`-XX:+UseParallelGC`): Usa múltiplas threads para operações de GC, reduzindo o tempo de pausa. As pausas Full GC ficaram entre 23 e 41 ms. Optimizado para throughput, sacrificando latência.

G1GC (`-XX:+UseG1GC`): Divide a heap em regiões e recolhe as mais rentáveis primeiro (*Garbage First*). As pausas Young ficaram entre 3 e 10 ms; as pausas Full (provocadas pelo `System.gc()` do benchmark) entre 7 e 10 ms. Mantém o heap em torno de 38–42 MB de forma consistente.

ZGC (`-XX:+UseZGC`): GC de baixa latência com recolha maioritariamente concorrente (sem pausas stop-the-world significativas). No entanto, o overhead de manutenção de

barreiras de escrita e leitura concorrentes aumentou o tempo sequencial de 22–24 ms para 32 ms, e o consumo de memória foi mais elevado (60–80 MB).

3.6.2 Resultados e Escolha

Tabela 3.1: Comparação de Garbage Collectors (sequencial / melhor paralelo)

GC	Tempo Seq. (ms)	Melhor (ms)	Pausa Max. (ms)	Mem. Média (MB)
Serial GC	23	5	69	38–56
Parallel GC	21	5	41	38–66
G1GC	22	5	11	38–43
ZGC	32	13	N/A	58–80

Configuração escolhida: G1GC. O G1GC apresentou o melhor equilíbrio entre latência de pausa (máximo de 11 ms) e throughput (tempo sequencial de 22 ms, idêntico ao Parallel GC). O Serial GC é inadequado para código paralelo pois as suas pausas longas interrompem todas as threads da aplicação. O ZGC, apesar de pausas mínimas, introduz overhead nas operações de leitura e escrita que prejudicou o desempenho global neste tipo de carga de trabalho.

Os parâmetros utilizados em produção:

```
-XX:+UseG1GC  
-XX:MaxGCPauseMillis=200  
-XX:G1HeapRegionSize=4m  
-Xms256m -Xmx2g
```

4 Concorrência e Sincronização

4.1 Estrutura das Fases

O algoritmo de equalização de histograma tem uma estrutura de fases (*pipeline barrier*):

1. **Fase 1** (paralelizável): Computação do histograma — os píxeis são independentes.
2. **Barreira**: Merge dos histogramas parciais — necessita que todas as threads da fase 1 tenham terminado.
3. **Fase 2** (sequencial): O histograma cumulativo é calculado em $O(256)$ — custo negligenciável.
4. **Fase 3** (paralelizável): Reescrita dos píxeis — independentes e em linhas disjuntas.

4.2 Thread Safety

Fase 1 (histograma): Cada thread opera sobre um array local `int[256]`. Não há estado compartilhado mutável. Após `join()/Future.get()/allOf().join()`, as escritas de todas as threads são visíveis na thread principal (garantia de *happens-before* da API de concorrência Java).

Merge: Executado exclusivamente na thread principal. Não requer sincronização.

Fase 3 (equalização): Cada thread escreve em linhas disjuntas do array `result`. O array `cumulative` é lido mas nunca escrito pelas threads, pelo que não há *data races*.

4.3 Estruturas de Dados e Mecanismos

Tabela 4.1: Mecanismos de sincronização por implementação

Implementação	Sincronização	Justificação
Multithreaded	<code>Thread.join()</code>	Barreira explícita entre fases
Thread Pool	<code>Future.get()</code>	Bloqueante por tarefa; recolhe resultado
Fork/Join	<code>fork/join, invokeAll</code>	Implícito no framework
CompletableFuture	<code>allOf().join()</code>	Composição declarativa de dependências

A opção por histogramas parciais por thread (sem `synchronized` nem `AtomicIntegerArray`) é intencional: elimina *contention* e falsa partilha de cache. O custo do merge (256 adições) é desprezável face ao trabalho paralelo.

5 Análise de Performance

5.1 Configuração do Ambiente

- **JVM:** Java 21, G1GC (`-Xms256m -Xmx2g`)
- **Imagem:** 860 × 1276 píxeis (1 097 360 píxeis)
- **Processadores:** 22 lógicos
- **Metodologia:** 3 execuções de aquecimento (JIT), média de 5 medições com `System.nanoTime()`

5.2 Resultados

Tabela 5.1: Resultados completos do benchmark (G1GC, imagem 860×1276)

Implementação	Tempo (ms)	Memória (MB)	Speedup
Sequential	24	38.20	1.00×
Multithreaded – 1 thread	22	39.97	1.09×
Multithreaded – 2 threads	18	39.76	1.33×
Multithreaded – 4 threads	10	39.85	2.40×
Multithreaded – 8 threads	14	40.71	1.71×
Multithreaded – 22 threads	12	40.39	2.00×
Thread Pool – 1 thread	22	38.35	1.09×
Thread Pool – 2 threads	13	38.57	1.85×
Thread Pool – 4 threads	9	39.58	2.67×
Thread Pool – 8 threads	7	39.49	3.43×
Thread Pool – 22 threads	6	40.13	4.00×
Fork/Join	7	39.49	3.43×
CompletableFuture – 1 thread	23	38.04	1.04×
CompletableFuture – 2 threads	13	38.67	1.85×
CompletableFuture – 4 threads	9	39.85	2.67×
CompletableFuture – 8 threads	7	39.67	3.43×
CompletableFuture – 22 threads	6	39.74	4.00×

5.3 Eficiência e Ganhos

A solução sequencial demora 24 ms. As melhores implementações paralelas atingem 6 ms, um *speedup* de 4×. Este valor está abaixo do máximo teórico de Amdahl — com 22 processadores, seria possível um speedup muito superior se o algoritmo fosse completamente paralelizável. A limitação deve-se a:

- **Fase 2 sequencial:** O histograma cumulativo ($O(256)$) é intrinsecamente sequencial e constitui uma barreira entre as fases 1 e 3.
- **Overhead de coordenação:** Para imagens de tamanho moderado (1M píxeis), o custo de criar/gerir threads, fazer *fork/join* ou lançar *CompletableFuture* representa uma fração relevante do tempo total.

- **Pressão de memória:** A imagem ocupa 38 MB. Com muitos threads, o acesso concorrente à memória pode introduzir contenção no bus de memória e *cache misses*.

5.4 Escalabilidade

Tabela 5.2: Escalabilidade com número de threads (Thread Pool e CompletableFuture)

Threads	Thread Pool		CompletableFuture	
	Tempo (ms)	Speedup	Tempo (ms)	Speedup
1	22	1.09×	23	1.04×
2	13	1.85×	13	1.85×
4	9	2.67×	9	2.67×
8	7	3.43×	7	3.43×
22	6	4.00×	6	4.00×

O *speedup* cresce de forma sub-linear com o número de threads, como previsto pela Lei de Amdahl. A partir de 8 threads, os ganhos diminuem significativamente — com 22 threads, o tempo (6 ms) é apenas marginalmente melhor do que com 8 threads (7 ms). Isto sugere que a fração paralelizável do algoritmo está próxima de saturar.

5.5 Overhead de Coordenação

Ao comparar o Multithreaded sem pool com o Thread Pool com 1 thread, o Thread Pool apresenta um overhead ligeiramente superior ao Sequential (22 ms vs. 24 ms). Isto deve-se ao custo de submissão de tarefas ao `ExecutorService`. A partir de 2 threads, o Thread Pool supera claramente o Multithreaded sem pool, evidenciando que a reutilização de threads compensa o overhead de gestão do pool quando há múltiplas fases de paralelismo.

O Fork/Join apresenta 3.43× de speedup sem necessidade de especificar o número de threads, delegando a decisão ao runtime. Para imagens maiores, este mecanismo de *work stealing* tornaria o Fork/Join ainda mais competitivo.

5.6 Análise de Memória

O consumo de memória mantém-se consistente entre implementações (38–41 MB), refletindo principalmente o tamanho da imagem em memória. O ligeiro aumento nas implementações paralelas (2 MB) deve-se aos arrays de histogramas parciais criados por thread. Não foram observadas fugas de memória — o `ExecutorService` é encerrado com `shutdown()` em `finally`, e o `ForkJoinPool` é fechado via *try-with-resources*.

5.7 Bottlenecks Identificados

1. **Fase sequencial obrigatória (histograma cumulativo):** Embora $O(256)$, introduz uma barreira que impede a fusão das fases 1 e 3 numa única passagem paralela.
2. **Granularidade de tarefas:** Para imagens de 1M píxeis e 22 threads, cada thread processa apenas 50k píxeis. O overhead relativo de lançar e sincronizar threads torna-se significativo.
3. **Criação de objetos Color:** A fase 3 cria um novo objeto `Color` por píxel (1M objetos por execução). Isto pressiona o GC e é um bottleneck independente do grau de paralelismo.

6 Conclusões

6.1 Síntese dos Resultados

Este projeto demonstrou que a paralelização da equalização de histograma produz ganhos reais mas limitados pela estrutura do algoritmo. O melhor *speedup* obtido foi de 4× (de 24 ms para 6 ms), com Thread Pool de 22 threads e `CompletableFuture` de 22 threads, numa máquina de 22 processadores.

O **Fork/Join** destacou-se pela ausência de configuração explícita de threads e pelo comportamento estável, sendo particularmente adequado para algoritmos de *divide-and-conquer* como o histograma. O **CompletableFuture** ofereceu código mais expressivo e declarativo sem penalidade de desempenho face ao Thread Pool. O **Multithreaded manual** demonstrou os desafios de gestão de threads e a importância de evitar contenção, mas ficou aquém do Thread Pool a partir de 4 threads.

6.2 Garbage Collector

O **G1GC** foi o GC mais equilibrado para esta carga de trabalho, com pausas máximas de 11 ms e tempo sequencial competitivo. O **ZGC**, apesar de pausas mínimas, introduziu overhead de barreira que penalizou todas as implementações. Para aplicações de processamento de imagem que criam muitos objetos `Color` por execução, o **G1GC** oferece o melhor compromisso entre latência e throughput.

6.3 Reflexão

A Lei de Amdahl explica o comportamento observado: com uma fração sequencial de 5% (histograma cumulativo + overhead fixo), o *speedup* máximo teórico é 15×. O *speedup* obtido de 4× indica que há ainda espaço para otimização — nomeadamente ao substituir os objetos `Color` por arrays de inteiros raw, eliminando a pressão no GC durante a fase 3.

Referências

- [1] R.C. Gonzalez and R.E. Woods. *Digital Image Processing*, 4th edition. Pearson, 2018.
- [2] Robert Sedgewick and Kevin Wayne. *Computer Science: An Interdisciplinary Approach*, 1st edition. Addison-Wesley Professional, 2016.
- [3] Brian Goetz et al. *Java Concurrency in Practice*. Addison-Wesley, 2006.
- [4] Oracle Corporation. *Fork/Join Framework*. Java Documentation, 2024. <https://docs.oracle.com/javase/tutorial/essential/concurrency/forkjoin.html>
- [5] Oracle Corporation. *CompletableFuture API*. Java SE 21 Documentation, 2024. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/CompletableFuture.html>